

SC4063: Network Security

Week 5 : Intrusion Detection

Chua Zong Fu





IDS vs IPS vs NSM

- **Intrusion Detection System (IDS):** Primarily focuses on deep content inspection and generating alerts based on known signatures of malicious behavior. It is often a reactive "burglar alarm".
- **Intrusion Prevention System (IPS):** Expands on IDS capabilities by actively preventing threats in real-time (e.g dropping packets or resetting connections) rather than just alerting.
- **Network Security Monitoring (NSM):** Distinct from traditional IDS; NSM focuses on the "collection, detection, and analysis" cycle. Unlike an IDS which acts as a "black box" generating alerts, NSM focuses on collecting data (full packet capture, netflow, session data) to provide context for the analyst.

An **IDS** is passive; it watches a copy of the traffic via a SPAN port or Tap and alerts while An **IPS** sits inline and can actively drop packets,. If IPS is so useful, *Why wouldn't we always use IPS?* False positives in an IPS block legitimate traffic, causing a Denial of Service, whereas an IDS false positive is just a nuisance alert.

While an IDS tells you you *bad* things are happening, NSM is "data-driven", telling you *everything* that has happened. IDS is a guard who yells when they see a thief; NSM is a CCTV camera recording everyone, allowing you to investigate *how* the thief got in later.



Sensor Deployment (Similar to Network Tap)

- **Placement Strategy:** Sensors must be placed at critical choke points, such as internet gateways and internal network segments (e.g DMZ, Server VLANs) to maximize visibility.
- **Components:** A sensor typically includes a packet decoder to parse protocols, a detection engine to match patterns, and an output mechanism for alerts/logs.

Sensors" must be placed at choke points, typically behind the firewall (to see what got in) and before critical servers (to see internal lateral movement).



Detection Methodologies

Signature-Based Detection :

- Compares observed traffic against a database of known attack signatures (rules).
- Looks for specific strings, hexadecimal sequences, or regular expressions within the packet payload.
- Highly effective against known threats but blind to zero-day attacks or modified exploits that do not match existing rules.

Anomaly-Based Detection:

- Establishes a "normal" baseline of network behavior and flags statistical deviations (e.g volume spikes, unusual port usage).
- Examines connectivity patterns, such as the ratio of bytes sent vs. received, to identify potential data exfiltration or beaconing.
- Capable of detecting unknown threats (zero-days) but prone to high false-positive rates if baselines are not well-tuned.

Protocol Verification:

- Detects when a specific protocol (e.g HTTP, DNS) violates standard RFC compliance, which often indicates an exploit attempt or a non-standard covert channel.

Signature-Based detection is like antivirus for the network. It looks for specific hex bytes or strings such as “Get /etc/passwd”.

Such detection approach fails against unknown attacks or zero days attack where no signatures are available yet.

Anomaly-Based identify the normal baseline and alerts on deviations. For example, if a printer usually sends 5MB of data a day but suddenly sends 5GB, that is an anomaly.

Such detection often generate a high false positive rate for chaotic network. Abnormal network behaviour is not always "malicious".

Protocol Verification detects attacks that violate standards. For example, a HTTP request that is 10,000 characters long, causing a buffer overflow due to its length. Detecting RFC violations captures many exploits without needing a specific signature.



IDS Tool : Snort, Suricata, Zeek, Security Onion

Snort and Suricata (Rule-Based):

- Provide a language to define what is considered malicious. Analysts must understand how to view, alter, and create rules to cover gaps where no vendor signatures exist (e.g zero-day exploits).
- Performs packet decoding, parse basic protocols, and execute detection algorithms on the packet stream.

Zeek (Formerly Bro):

- Unlike Snort, Zeek generates logs (e.g conn.log, http.log, ssl.log) organizing traffic by protocol, which can then be monitored and alerted upon, using a SIEM.
- Supports file extraction (calculating SHA256 hashes) and integrates community ID hashing for correlating flows across tools.

Security Onion:

- A Linux distribution that integrates Snort/Suricata, Zeek, and analysis tools (like Kibana) into a cohesive Network Security Monitoring platform.

Snort & Suricata are rule-based engines. Suricata is often preferred in modern high-speed networks because it is multi-threaded whereas legacy Snort is single-threaded.

Zeek is *not* primarily for alerts; it is a "transaction logger." It turns raw packets into tab-separated logs like http.log and conn.log. It is scriptable, allowing for complex logic to be implemented. For example Alert if a user downloads an EXE file and then immediately connects to a rare IP.

Security Onion is the "all-in-one" Linux distribution for network forensic. It integrates Snort, Suricata, Zeek and Kibana into a single platform.



IDS Analysis

1. Alert Triage

- Analysts are often overwhelmed by alerts. Triage involves prioritizing alerts based on severity (e.g. Root Intrusion vs. Reconnaissance) and distinguishing false positives from true positives. Alerts alone are insufficient; analysts must pivot to packet captures (PCAP) or transaction logs to verify if an attack was successful.

2. Flow Analysis

- Summarizes traffic (Source IP, Dest IP, Ports, Bytes) without storing full payloads. Essential for analyzing high-bandwidth networks where full capture is prohibitive.

3. Long-Tail Analysis (Hunting for Least Frequent Occurrences)

- Instead of focusing on top talkers, analysts filter for unique, solitary connections (e.g. a single User-Agent string appearing once) which may indicate stealthy C2 channels.

Alert Triage. An un-tuned IDS generates thousands of alerts. There is a need to perform triage to establish a priority of how we deal with threats identified. High priority isn't just "high severity". it's a high confidence alert on a threat detected, targeting a critical asset.

Flow Analysis : Sometimes you don't have full packet capture because it takes too much storage. NetFlow is like a "phone bill" that tells you who talked to whom and for how long, but not *what* they said. It is essential for high-level visibility.

Long-Tail Analysis. This is a crucial hunting technique. Instead of looking for the "Top 10" talkers, look for the "Bottom 10" (Least Frequency of Occurrence). A malware beacon might only pulse once a day. This is how you find the "needle in the haystack" that doesn't trigger volumetric thresholds,,.



IDS Analysis

4. JA3 Fingerprinting

- Profiling client applications (e.g malware vs. Chrome) based on the unencrypted TLS 'Client Hello' packet parameters. Zeek can log these hashes ('JA3' and 'JA3S') to identify malicious tools even inside encrypted tunnels.

5. Certificate Forensics

- Analyzing x.509 certificates for anomalies, such as validity dates or self-signed issuers, often found in malware campaigns.

Modern malware uses TLS/HTTPS. Traditional signatures fail because they can't read the payload.

JA3 Fingerprinting. We can detect malware in encrypted traffic *without* decrypting the payload. To do so, we look at the *way* the client says "Hello". Different applications (Chrome vs. Python script vs. Malware) use different ciphers in different orders. This creates a unique hash (JA3) we can use for blacklisting.

File and Certificate Extraction. Tools like Zeek can automatically carve files such as Certificates, EXEs or PDFs from the stream for malware analysis. This connects network forensics to host forensics.



IDS Evasion

- **Insertion:** An attacker sends packets that the **IDS accepts** but the target host rejects, causing the IDS to see a "malicious" stream that the target never processes. E.g Packets with bad checksum, or TTL that doesn't allow the packet to reach the host.
- **Evasion:** An attacker sends packets that the **IDS rejects** but the target host accepts, allowing the attack to slip past the sensor. E.g Packets with overlapping segments
- **Fragmentation:** Splitting malicious payloads across tiny IP fragments to confuse the reassembly process of the IDS.
- **TTL Manipulation:** Using Time-To-Live fields to ensure certain packets reach the IDS but expire before reaching the target (or vice versa), desynchronizing the view of the data stream.
- **Encryption:** Tunneling attacks through encrypted protocols (HTTPS, SSH) to bypass payload inspection.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Attackers know you are watching. They will try to trick the IDS.

Fragmentation Attacks: Attackers chop a malicious payload into tiny packet fragments and send them across the wire. If the IDS doesn't reassemble them, it sees innocent fragments.

If the Victim reassembles them, they get hacked. Modern IDSs *must* perform stateful reassembly to catch this, but this requires more RAM and compute resource, driving up the cost per appliance.

Insertion: An attacker sends a packet with a short Time-To-Live (TTL). It reaches the IDS (alerting it) but dies before reaching the target. The IDS thinks an attack happened, but the target never saw it. This creates "noise" to hide real attacks.

Evasion: An attacker sends packet that the IDS rejects but the host accepts. An Example is packet with overlapping fragment segments.

Insertion and Evasion attacks relies on discrepancies in how the IDS and the target operating system (e.g Windows vs. Linux) reassemble packet fragments.

Encryption : More frequently, attackers are using HTTPS as the most common evasion today. If you don't have a "Break and Inspect" proxy, you are blind to the payload.